CampusOS API Guide

Everything you need to build a CampusOS frontend and connect it to the API. You get the base URL and your team's admin email and OTP from the organisers.

You can also try every call in the browser at `{baseUrl}/docs`.

Contents

1. How it works (includes sample data)
2. Basics you need everywhere
3. Login and accounts
4. FindIt: Lost & Found
5. Notice Hub: Campus notices
6. DocDesk: Document requests
7. FixIt: Maintenance tickets
8. Error codes
9. Limits
10. All endpoints at a glance

Each feature section lists the **core** APIs first and then the ⭐ **bonus** APIs.

---

1. How it works

Your team is one **institution** with its own email domain (for example `@team1.com`). Every account belongs to that domain, and you only ever see your own institution's data.

Roles

Role	Who	Created by
`admin`	One per team, given by the organisers	The organisers
`student`	Students	The admin
`faculty`	Teachers	The admin
`office_staff`	Office staff. Their `dept` decides extra powers (see below)	The admin

Two office departments have special jobs:

Office staff with `dept`	Runs
`Student Affairs`	DocDesk: document types, answering document requests
`Campus Maintenance`	FixIt: issue types, working on tickets

The department name ignores case and a trailing "dept", so `student affairs` and `Student Affairs Dept` also work. See Sample data for more department names.

Getting started

1. Log in as the **admin** with the email and OTP from the organisers.
2. Create a few users: a student, a faculty member, a Student Affairs staff member and a Campus Maintenance staff member. Each new user gets an **OTP** in the response.
3. Log each user in with their email and OTP, and keep one token per user so you can test every role.

Sample data you can use

Use these values when creating test data, or offer them as dropdown options in your UI. Only the two office departments in **bold** are special: the API looks for them by name. Every other value is free text.

Student departments (`dept` for students)

Value	Meaning
`CSE`	Computer Science & Engineering
`ISE`	Information Science & Engineering
`ECE`	Electronics & Communication Engineering
`MBA`	Business Administration

Office departments (`dept` for office staff)

Value	Runs	Example `designation`
`Student Affairs`	DocDesk (document requests)	`Section Officer`, `Clerk`
`Campus Maintenance`	FixIt (maintenance tickets)	`Supervisor`, `Electrician`
`Accounts`	normal office staff	`Accountant`, `Cashier`
`Library`	normal office staff	`Librarian`, `Library Assistant`

Any office staff member can post found items in FindIt and notices in Notice Hub, whatever their department. Spell the two special departments exactly as shown (case doesn't matter). Otherwise the user gets `403 FORBIDDEN` in DocDesk or FixIt.

Faculty designations (`designation` for faculty): `Professor`, `Associate Professor`, `Assistant Professor`, `Head of Department`.

Document types (DocDesk, created by Student Affairs with `POST /api/v1/docdesk/doc-types`)

`name`	`description`
`Bonafide Certificate`	Proof that you are a student here, e.g. for a bank or passport
`Transcript`	All semester marks on one sheet
`Scholarship Form`	Scholarship form to fill in and return (send it as the answer's document)
`Experience Certificate`	For staff and faculty

Issue types (FixIt, created by Campus Maintenance with `POST /api/v1/fixit/issue-types`)

`name`	`description`
`Electrical`	Fans, lights, switches, sockets
`Plumbing`	Taps, leaks, blocked drains, washrooms
`Furniture`	Broken benches, chairs, desks, doors
`IT & Network`	Wi-Fi, projectors, lab computers

Create document types and issue types **before** testing requests and tickets. Every request needs a `doc_type_id` and every ticket needs an `issue_type_id`.

---

2. Basics you need everywhere

Sending the token

Every endpoint except login and health needs this header:

```
Authorization: Bearer <access_token>
```

Request formats

- Most endpoints take a **JSON** body (`Content-Type: application/json`).
- Endpoints that accept a file take `multipart/form-data` (form fields + file). Each endpoint below says which one it uses.
- In form-data, send booleans as the text `true` or `false`.

Ids

The API creates every id. The prefix tells you what the id is for:

Prefix	Thing	Prefix	Thing
`FND-`	found post	`DTY-`	document type
`CLM-`	claim	`DRQ-`	document request
`LST-`	lost request	`ITY-`	issue type
`NTC-`	notice	`TKT-`	ticket
`DBT-`	doubt		

Sending the wrong kind of id returns `422`.

Lists and paging

Every list comes back newest first, in this shape:

```
{ "items": [ ... ], "count": 25, "next_cursor": "FND-8206740152499-46fa2f79" }
```

- `?limit=` sets the page size: 1–100, default 25.
- To get the next page, call again with `?cursor=<next_cursor>`.
- `next_cursor: null` means you have reached the last page.

Editing (PATCH)

- Send **only the fields you want to change**. Everything else stays as it is.
- Sending no fields, or a field that doesn't exist, returns `422`.
- Optional text fields can be cleared: send `null` in JSON, or an empty value in form-data.

- When a record has a file, a new file in the PATCH **replaces** the old one. To delete the file, send `remove_<name>=true` (for example `remove_photo=true`).

Files

- Responses show a file as `{ "file_id"/"image_id"/"attachment_id", "filename", "content_type", "size_bytes", "url" }`.
- The `url` is a direct link that **expires after about 15 minutes**. It works for `<img src>` and quick previews.
- For a download button, use the feature's download endpoint with the token. Those always work.
- The API checks the file type from the file's content, so renaming a file doesn't change its type.

Dates

- Dates like `found_on` or `expected_by` are `YYYY-MM-DD`.
- `publish_at` (Notice Hub) is a date-time. If you send no timezone, it is read as **India time (IST)**.
- Every time in a response is in **UTC** (ends in `Z`). Convert it to IST for display.
- Optional fields with no value are **left out** of responses, so check whether they exist before using them.

Errors

Every error has the same shape:

```
{
  "error": { "code": "POST_NOT_ACTIVE", "message": "This post is archived and no longer takes claims", "details": null },
  "request_id": "6f1c..."
}
```

Base your UI logic on `error.code`. You can show `message` to users, but its wording may change. For `422 VALIDATION_ERROR`, `details` lists each bad field:

```
{ "error": { "code": "VALIDATION_ERROR", "message": "Request validation failed. Check 'details' for the offending fields.",
  "details": [ { "field": "message", "in": "body", "message": "String should have at least 10 characters", "type": "string_too_short" } ] } }
```

All error codes are listed in section 8.

---

3. Login and accounts

3.1 Log in

Who	Endpoint
Admin	`POST /api/v1/auth/admin/login`
Student, faculty, office staff	`POST /api/v1/auth/login`

Body (JSON):

```
{ "email_id": "asha@team1.com", "otp": "730792" }
```

Response `200`:

```
{
  "access_token": "eyJhbGciOi...",
  "token_type": "bearer",
  "expires_in": 43200,
  "profile": { "role": "student", "team_id": "team1", "name": "Asha Kumar", "email_id":
"asha@team1.com", "...": "..." }
}
```

- `expires_in` is in seconds. After that the API returns `401 TOKEN_EXPIRED`, so send the user back to the login screen.
- Use `profile.role` (and `profile.dept` for office staff) to decide what the user sees.
- Possible errors: `401 INVALID_CREDENTIALS`, `400 UNKNOWN_INSTITUTION` (unknown email domain), `400 USE_ADMIN_LOGIN` (the admin used the user login), `429 RATE_LIMITED` (too many wrong attempts, wait and retry).

3.2 My profile

Endpoint	Who	Returns
`GET /api/v1/users/me`	student, faculty, office staff	User profile (below)
`GET /api/v1/admin/me`	admin	`{ role, team_id, institution_name, name, email_id, phone_number }`

User profile:

```
{
  "role": "office_staff", "team_id": "team1", "name": "Ravi Shetty", "email_id": "ravi@team1.com",
  "phone_number": "9876543211", "dept": "Student Affairs", "designation": "Accountant",
  "created_at": "2026-10-30T08:00:00Z", "updated_at": "2026-10-30T08:00:00Z"
}
```

Only students have `usn`. Only students and office staff have `dept`. Only faculty and office staff have `designation`.

Users can read their profile but can't change it. Only the admin can.

3.3 Create users (admin)

`POST /api/v1/admin/users` · **admin** · JSON → `201`

The `role` field decides which other fields you send. Every field shown is required, and sending a field that doesn't belong to the role returns `422`.

Student

```
{
  "role": "student",
  "name": "Asha Kumar",
  "usn": "1CF24CS001",
  "dept": "CSE",
  "email_id": "asha@team1.com",
  "phone_number": "9876543210"
}
```

Office staff (`dept` decides DocDesk / FixIt powers, see Sample data)

```
{
  "role": "office_staff",
  "name": "Ravi Shetty",
  "dept": "Student Affairs",
  "designation": "Accountant",
  "email_id": "ravi@team1.com",
  "phone_number": "9876543211"
}
```

```
{
  "role": "office_staff",
  "name": "Kumar Gowda",
  "dept": "Campus Maintenance",
  "designation": "Supervisor",
  "email_id": "kumar@team1.com",
  "phone_number": "9876543215"
}
```

Faculty

```
{
  "role": "faculty",
  "name": "Dr. Meera Rao",
  "designation": "Professor",
  "email_id": "meera@team1.com",
  "phone_number": "9876543212"
}
```

Field rules:

Field	Rule
`name`	2–100 chars
`usn`	5–20 letters/digits, stored in uppercase
`dept` , `designation`	2–60 chars
`email_id`	must use **your team's domain**, stored in lowercase
`phone_number`	10–15 digits, optional leading `+`

Response: the user's profile plus `otp` (their login OTP) and `created_by` :

```
{ "role": "student", "name": "Asha Kumar", "email_id": "asha@team1.com", "...": "...", "otp": "730792", "created_by": "admin@team1.com" }
```

Errors: `409 ALREADY_EXISTS` (the email is already registered), `422 EMAIL_DOMAIN_MISMATCH` (wrong domain), `422 VALIDATION_ERROR` (missing or extra fields).

3.4 Manage users (admin)

Method	Endpoint	What it does
GET	`/api/v1/admin/users?role=&limit=25&cursor=`	List users, including their OTPs. `role` is optional: `student` , `faculty` or `office_staff`
GET	`/api/v1/admin/users/{email_id}`	Get one user, including their OTP
PATCH	`/api/v1/admin/users/{email_id}`	Change details (JSON): any of `name` , `usn` , `dept` , `designation` , `email_id` , `phone_number` . Fields must belong to the user's role (otherwise `422 FIELD_NOT_ALLOWED_FOR_ROLE`), and the role can't change
POST	`/api/v1/admin/users/{email_id}/regenerate-otp`	Give the user a new OTP. The old one stops working
DELETE	`/api/v1/admin/users/{email_id}`	Delete the user. Returns `{ "message": "..." }`

---

4. FindIt: Lost & Found

Base path: `/api/v1/findit`

Office staff post items they found. The owner **claims** an item, and staff **resolve** the post by choosing the right claim.

Who can do what

Action	Office staff	Faculty	Student	Admin
Post / edit / archive / delete found items	✅			
Browse found items	✅	✅	✅	✅
Claim an item, withdraw own claim	✅	✅	✅	✅
See all claims on a post	✅	own	own	own
Resolve a post	✅			
⭐ Report a lost item	✅	✅	✅	
⭐ See lost requests	everyone's	own	own	

What the data looks like

Found post

```
{
  "post_id": "FND-8206740152499-46fa2f79",
  "title": "Blue water bottle",
  "description": "Steel, blue lid",
  "location_found": "Library 2nd floor",
  "found_on": "2026-10-29",
  "images": [ { "image_id": "IMG-3c9e0a7f1b2d4e65", "url": "https://...", "content_type": "image/jpeg" }
],
  "status": "active",
  "resolved": false,
  "created_by": "ravi@team1.com",
  "created_at": "2026-10-30T08:00:00Z",
  "updated_at": "2026-10-30T08:00:00Z"
}
```

Office staff also get `resolution` once a post is resolved. Other users only see `resolved: true`, never who got the item.

Claim

```
{
  "claim_id": "CLM-a71197e76b312ab0",
  "post_id": "FND-8206740152499-46fa2f79",
  "claimant_email": "asha@team1.com", "claimant_name": "Asha Kumar",
  "claimant_role": "student", "claimant_phone_number": "9876543210",
  "message": "My name is written under the lid",
  "status": "active",
  "decision": "pending",
  "created_at": "...", "updated_at": "..."
}
```


`decision` is `pending`, `approved` or `rejected`.

Lost request ⭐


```
{
  "request_id": "LST-8206740149371-5377e979",
  "title": "Black umbrella",
  "description": "Folding, wooden handle",
  "location_lost": "Canteen",
  "lost_on": "2026-10-28",
  "status": "active",
  "created_by": "asha@team1.com", "created_by_name": "Asha Kumar",
  "created_by_role": "student", "created_by_phone_number": "9876543210",
  "created_at": "...", "updated_at": "..."
}
```


Core APIs

Post a found item

`POST /posts` · **office staff** · form-data → `201` found post

Field	Required	Notes
`title`	✅	3–100 chars
`description`		up to 2000 chars
`location_found`		2–120 chars
`found_on`		`YYYY-MM-DD`, not in the future
`images`		file. Repeat the field for more images: up to 5, JPEG/PNG/WebP, 5 MB each

Browse found items

Method	Endpoint	Who	Returns
GET	`/posts?limit=25&cursor=`	everyone	list of active found posts
GET	`/posts/{post_id}`	everyone	one found post

Edit or delete a found item

Method	Endpoint	Who	Notes
PATCH	`/posts/{post_id}`	office staff	form-data, any of: `title`, `description`, `location_found`, `found_on`, `images` (replaces **all** images), `remove_images=true`
DELETE	`/posts/{post_id}`	office staff	deletes the post with its images and claims. Fails with `409 POST_HAS_PENDING_CLAIMS` while a claim still waits for a decision

Claims

Method	Endpoint	Who	Notes
POST	`/posts/{post_id}/claims`	everyone	JSON `{ "message": "proof it's yours, 10–1000 chars" }` → `201` claim. Only one claim per person per post
GET	`/posts/{post_id}/claims`	everyone	office staff see every claim on the post, others only their own
GET	`/claims/me?limit=25&cursor=`	everyone	my claims on all posts
GET	`/claims/{claim_id}`	claimant, office staff	one claim
DELETE	`/claims/{claim_id}`	claimant	withdraw, only while `pending`

Resolve a post (hand the item over)

Method	Endpoint	Who	Notes
POST	`/posts/{post_id}/resolution`	office staff	JSON `{ "claim_id": "CLM-...", "note": "Collected with ID card" }` (`note` optional) → `201` found post
GET	`/posts/{post_id}/resolution`	office staff, the approved claimant	`{ claim_id, claimant_email, note, resolved_by, resolved_at, updated_at }`
PATCH	`/posts/{post_id}/resolution`	office staff	JSON `{ "claim_id"?, "note"? }`: pick a different claim or change the note
DELETE	`/posts/{post_id}/resolution`	office staff	undo the resolution. The post becomes active again and every claim goes back to pending

What resolving does:

Event	Post	Claims on it
Resolve	`archived`, `resolved: true`	the chosen claim is `approved`, the others `rejected`, and all are `archived`
Change resolution	stays resolved	decisions are worked out again for the new choice

Event	Post	Claims on it
Undo resolution	`active`, `resolved: false`	all `active`, `pending`

⭐ Bonus APIs

⭐ Lost requests

Students, faculty and office staff report something they lost, and office staff can see every report.

Method	Endpoint	Who	Notes
POST	`/lost-requests`	faculty, office staff, student	JSON → `201` lost request
GET	`/lost-requests?status=active&mine=false&limit=25&cursor=`	faculty, office staff, student	office staff see everyone's (`mine=true` shows only their own), others see their own
GET	`/lost-requests/{request_id}`	owner, office staff	one lost request
PATCH	`/lost-requests/{request_id}`	owner	JSON, any of: `title`, `description`, `location_lost`, `lost_on`, `status`
DELETE	`/lost-requests/{request_id}`	owner	delete it

Create body:

```
{ "title": "Black umbrella", "description": "Folding, wooden handle", "location_lost": "Canteen", "lost_on": "2026-10-28" }
```

Only `title` (3–100 chars) is required. `lost_on` can't be in the future.

⭐ Active / archived status

Everything starts `active`. Lists show only `active` items unless you ask for `?status=archived`.

What you want	Call
Archive a found post without resolving it	`PATCH /posts/{post_id}` form-data `status=archived`. Its claims are archived too
Reopen an archived post	`PATCH /posts/{post_id}` `status=active`. Its claims become active again. Doesn't work for a resolved post (`409 POST_RESOLVED`): undo the resolution instead
Show archived found posts	`GET /posts?status=archived`
Mark my lost item as found	`PATCH /lost-requests/{request_id}` JSON `{ "status": "archived" }`
Reactivate my lost request	`PATCH /lost-requests/{request_id}` JSON `{ "status": "active" }`

What you want	Call
Show archived lost requests	`GET /lost-requests?status=archived`

---

5. Notice Hub: Campus notices

Base path: `/api/v1/noticehub`

Admin, faculty and office staff post notices. Readers ask **doubts** on a notice, and the publisher answers each doubt with one **response**.

Who can do what

Action	Admin	Faculty	Office staff	Student
Post a notice	✅	✅	✅	
Read notices, download attachments	✅	✅	✅	✅
Edit / pin / schedule / archive a notice	own	own	own	
Delete a notice	any	own	own	
Ask a doubt		✅	✅	✅
Edit own doubt (until it is answered)		✅	✅	✅
Delete a doubt	any	own	own	own
Respond to a doubt	any notice	own notices	own notices	
Edit / remove a response	any	own	own	
⭐ See scheduled notices	all	own	own	

What the data looks like

Notice

```
{
  "notice_id": "NTC-8206740152499-46fa2f79",
  "title": "Mid-term timetable",
  "body": "Mid-terms start on 10 November.",
  "status": "active",
  "pinned": false,
  "publish_at": "2026-10-30T08:00:00Z",
  "attachment": { "attachment_id": "ATT-...", "filename": "timetable.pdf", "content_type":
"application/pdf", "size_bytes": 182311, "url": "https://..." },
  "created_by": "meera@team1.com", "created_by_name": "Dr. Meera Rao", "created_by_role": "faculty",
  "created_at": "...", "updated_at": "..."
}
```

`status` is `scheduled`, `active` or `archived`. `attachment` is left out when there is none.

Doubt (the response is included once the doubt is answered)

```
{
  "doubt_id": "DBT-...",
  "notice_id": "NTC-...",
  "question": "Is the lab exam in the same week?",
  "asked_by": "asha@team1.com", "asked_by_name": "Asha Kumar", "asked_by_role": "student",
  "answered": true,
  "response": {
    "message": "No, lab exams are the week after.",
    "responded_by": "meera@team1.com", "responded_by_name": "Dr. Meera Rao", "responded_by_role":
"faculty",
    "responded_at": "...", "updated_at": "..."
  },
  "created_at": "...", "updated_at": "..."
}
```


Core APIs

Post a notice

`POST /notices` · **admin, faculty, office staff** · form-data → `201` notice

Field	Required	Notes
`title`	✅	3–150 chars
`body`	✅	1–5000 chars
`pinned`		⭐ `true` / `false`, default `false`
`publish_at`		⭐ schedule for later, e.g. `2026-11-03T09:00:00` (IST). Leave it out to publish now
`attachment`		⭐ one PDF, JPEG or PNG file, up to 10 MB

Read, edit, delete notices

Method	Endpoint	Who	Notes
GET	`/notices?limit=25&cursor=&mine=false`	everyone	active notices. `mine=true` shows only notices you posted
GET	`/notices/{notice_id}`	everyone	one notice
PATCH	`/notices/{notice_id}`	its publisher	form-data, any of: `title`, `body` (plus the ⭐ fields below)
DELETE	`/notices/{notice_id}`	its publisher, admin	deletes the notice with its attachment and doubts

Doubts

Method	Endpoint	Who	Notes
POST	`/notices/{notice_id}/doubts`	faculty, office staff, student	JSON `{ "question": "5–1000 chars" }` → `201` doubt. Only on active notices
GET	`/notices/{notice_id}/doubts?answered=&mine=false&limit=25&cursor=`	everyone	doubts with their responses. `answered=false` shows the unanswered ones (handy for the publisher's inbox)
GET	`/doubts/{doubt_id}`	everyone	one doubt
PATCH	`/doubts/{doubt_id}`	the asker	JSON `{ "question": "..." }`, only until it is answered
DELETE	`/doubts/{doubt_id}`	the asker, admin	delete it

Responses

Each doubt has **at most one** response. Body (JSON): `{ "message": "1–2000 chars" }`.

Method	Endpoint	Who	Notes
POST	`/doubts/{doubt_id}/response`	the notice's publisher, admin	→ `201` doubt with its response. Fails with `409 ALREADY_ANSWERED` if the doubt already has a response
GET	`/doubts/{doubt_id}/response`	everyone	just the response
PATCH	`/doubts/{doubt_id}/response`	the responder, admin	change the message
DELETE	`/doubts/{doubt_id}/response`	the responder, admin	remove it. The doubt becomes unanswered again

⭐ Bonus APIs

⭐ Scheduled notices

A notice with a future `publish_at` is `scheduled`. Students can't see it and nobody can ask doubts on it. When `publish_at` arrives, it becomes `active` automatically.

What you want	Call
Schedule a notice	`POST /notices` with `publish_at=2026-11-03T09:00:00`
See my scheduled notices (the admin sees everyone's)	`GET /notices?status=scheduled`
Change the time	`PATCH /notices/{notice_id}` with a new `publish_at`. Only works while the notice is still scheduled (`409 NOTICE_ALREADY_PUBLISHED`)
Cancel it	`DELETE /notices/{notice_id}`

`publish_at` without a timezone is IST. `2026-11-03T09:00:00`, `2026-11-03T09:00:00+05:30` and `2026-11-03T03:30:00Z` all mean 9:00 AM IST. Responses give it in UTC.

⭐ Active / archived status

What you want	Call
Archive a notice (still readable with its doubts, but takes no new doubts)	`PATCH /notices/{notice_id}` `status=archived`
Restore it	`PATCH /notices/{notice_id}` `status=active`. If its `publish_at` is still in the future, it goes back to `scheduled`
Show archived notices	`GET /notices?status=archived`

The list's `status` filter takes `active` (default), `archived` or `scheduled`.

⭐ Pinning

What you want	Call
Pin when posting	`POST /notices` with `pinned=true`
Pin or unpin later	`PATCH /notices/{notice_id}` with `pinned=true` or `pinned=false`
Only pinned notices (e.g. a top banner)	`GET /notices?pinned=true`
Only unpinned notices	`GET /notices?pinned=false`

⭐ Attachments and download

One attachment per notice: **PDF, JPEG or PNG, up to 10 MB**.

What you want	Call
Attach when posting	`POST /notices` with an `attachment` file
Add or replace later	`PATCH /notices/{notice_id}` with an `attachment` file
Remove it	`PATCH /notices/{notice_id}` with `remove_attachment=true`
Download	`GET /notices/{notice_id}/attachment` · everyone · returns the file itself

---

6. DocDesk: Document requests

Base path: `/api/v1/docdesk`

Users request official documents (bonafide certificate, transcript, ...). **Student Affairs** staff keep the list of document types, set turnaround times and answer requests, sometimes with a form to fill in.

Who can do what

Action	Student Affairs	Other office staff	Faculty	Student	Admin
Request a document, edit / withdraw own request while pending	✅	✅	✅	✅	
See requests	everyone's	own	own	own	everyone's
Answer, change or withdraw an answer	✅				
Acknowledge own answered request	✅	✅	✅	✅	
⭐ See document types	✅	✅	✅	✅	✅
⭐ Create / edit / archive / delete document types	✅				
⭐ Set turnaround time (one or bulk)	✅				
⭐ Download the document sent with the answer	any	own	own	own	any
⭐ Upload the filled-in form	own	own	own	own	
⭐ Download the filled-in form	any	own	own	own	any

The admin can **only read** here. Any create, edit or upload call from the admin returns `403`.

Request status



Status	Requester can	Student Affairs can
`pending`	edit, withdraw	set the turnaround time, answer
`answered`	download the document, upload the filled-in form, acknowledge	change the answer or its document, withdraw the answer
`acknowledged`	view, download	view, download

The answer is free text, so it covers every outcome: "Ready, collect it after 2pm", "Not issued: fee receipt missing", or "Fill in the attached form and upload it back".

What the data looks like

Document type ⭐

```
{
  "doc_type_id": "DTY-8206727421084-20448c93",
  "name": "Bonafide Certificate",
  "description": "Proof that you study here",
  "status": "active",
  "created_by": "ravi@team1.com", "created_at": "...", "updated_at": "..."
}
```

Request

```
{
  "request_id": "DRQ-8206727421084-20448c93",
  "doc_type_id": "DTY-...", "doc_type_name": "Bonafide Certificate",
  "reason": "Needed for the scholarship application",
  "status": "answered",
  "tat": { "expected_by": "2026-11-05", "note": "Waiting for signature", "updated_by": "ravi@team1.com",
"updated_at": "..." },
  "answer": { "message": "Fill in the attached form and upload it back", "answered_by":
"ravi@team1.com", "answered_by_name": "Ravi Shetty", "answered_at": "...", "updated_at": "..." },
  "document":    { "file_id": "DOC-...", "filename": "Scholarship_Form.docx", "content_type":
"application/vnd.openxmlformats-officedocument.wordprocessingml.document", "size_bytes": 48211, "url":
"https://..." },
  "filled_form": { "file_id": "...", "filename": "Scholarship_Form_filled.pdf", "content_type":
"application/pdf", "size_bytes": 90211, "url": "https://..." },
  "acknowledged_at": null,
  "requested_by": "asha@team1.com", "requested_by_name": "Asha Kumar", "requested_by_role": "student",
  "requested_by_dept": "CSE", "requested_by_phone_number": "9876543210",
  "created_at": "...", "updated_at": "..."
}
```

`tat`, `answer`, `document`, `filled_form` and `acknowledged_at` are left out until they exist.

Core APIs

Request a document

`POST /requests` · **faculty, office staff, student** · JSON → `201` request

```
{ "doc_type_id": "DTY-8206727421084-20448c93", "reason": "Needed for the scholarship application" }
```

`reason` is 5–1000 chars. Get `doc_type_id` from `GET /doc-types` (⭐ below, or show a fixed list if you skip the bonus). An archived type returns `409 DOC_TYPE_ARCHIVED`.

Track, edit, withdraw requests

Method	Endpoint	Who	Notes
GET	`/requests?status=&doc_type_id=&mine=false&limit=25&cursor=`	everyone	Student Affairs and the admin see all requests, others see their own. `status`: `pending`, `answered`, `acknowledged`. `mine=true` lets Student Affairs see only their own
GET	`/requests/{request_id}`	requester, Student Affairs, admin	one request
PATCH	`/requests/{request_id}`	requester	JSON, any of: `doc_type_id`, `reason`. Only while pending
DELETE	`/requests/{request_id}`	requester	withdraw, only while pending

Answer a request

Method	Endpoint	Who	Notes
POST	`/requests/{request_id}/answer`	Student Affairs	form-data: `message` (required, 1–2000 chars), `document` (⭐ optional file) → `201` request, now `answered`
PATCH	`/requests/{request_id}/answer`	Student Affairs	form-data, any of: `message`, `document`, `remove_document=true`. Allowed until the request is acknowledged
DELETE	`/requests/{request_id}/answer`	Student Affairs	withdraw the answer, with its document and any filled-in form. The request goes back to `pending`

Acknowledge

`POST /requests/{request_id}/acknowledge` · **requester** · no body → request with status `acknowledged`. The request is now closed.

⭐ Bonus APIs

⭐ Fetch document types

Use these to fill the "Document type" dropdown on the request form.

Method	Endpoint	Who	Notes
GET	`/doc-types?status=active&limit=100&cursor=`	everyone	`status`: `active` (default) or `archived`
GET	`/doc-types/{doc_type_id}`	everyone	one type

⭐ Manage document types (Student Affairs)

Method	Endpoint	Body (JSON)
POST	`/doc-types`	`{ "name": "Bonafide Certificate", "description": "Proof that you study here" }` (more in Sample data). `name` 3–80 chars and unique, `description` optional → `201`
PATCH	`/doc-types/{doc_type_id}`	any of: `name`, `description`, `status`. Use `"status": "archived"` to stop new requests for a type and `"active"` to bring it back
DELETE	`/doc-types/{doc_type_id}`	delete the type

A duplicate name returns `409 ALREADY_EXISTS`. Anyone else returns `403 FORBIDDEN`.

⭐ Turnaround time (one or many)

The turnaround time (TAT) tells the requester when to expect the document. Only Student Affairs can set it, and only on **pending** requests. `expected_by` is `YYYY-MM-DD` and can't be before today.

One request: `PUT /requests/{request_id}/tat` → updated request. Sending it again replaces the old value.

```
{ "expected_by": "2026-11-05", "note": "Waiting for the principal's signature" }
```

Many at once: `POST /requests/bulk-tat` → `{ "items": [ ...updated requests... ], "count": 2 }`

```
{
  "request_ids": ["DRQ-8206727421084-20448c93", "DRQ-8206727398811-0aa1f3c2"],
  "expected_by": "2026-11-05",
  "note": "Batch sent for signature"
}
```

- Send 1–100 ids. Repeated ids are ignored.
- **All or nothing:** if any id doesn't exist (`404`) or isn't pending any more (`409 REQUEST_NOT_PENDING`), nothing is changed. `error.details.request_ids` lists the ids that failed, so you can unselect them and retry.

UI idea: checkboxes on the pending list (`GET /requests?status=pending`), a date picker, and one "Set TAT" button.

⭐ Send a document and download it

Student Affairs can attach a document to the answer: **PDF or Word (.docx), up to 10 MB**.

What you want	Call
Answer with a document	`POST /requests/{request_id}/answer` form-data `message` + `document` file
Replace the document	`PATCH /requests/{request_id}/answer` with a new `document` file

What you want	Call
Remove the document	`PATCH /requests/{request_id}/answer` with `remove_document=true` (this also removes the filled-in form)
Download it	`GET /requests/{request_id}/document` · requester, Student Affairs, admin · returns the file

⭐ Fill in and upload back

The requester fills in the document (on a computer, or prints it, fills it and takes a photo) and uploads it back: **PDF, Word (.docx), JPEG or PNG, up to 10 MB**.

Method	Endpoint	Who	Notes
PUT	`/requests/{request_id}/filled-form`	requester	form-data with one `file`. Uploading again replaces it → updated request
GET	`/requests/{request_id}/filled-form`	requester, Student Affairs, admin	download it
DELETE	`/requests/{request_id}/filled-form`	requester	remove it (only before acknowledging)

This only works when the request is `answered` **and** the answer has a document. Otherwise you get:

Situation	Error
Request still pending	`409 NOT_ANSWERED`
Answer has no document	`409 NO_DOCUMENT`
Request already acknowledged	`409 ALREADY_ACKNOWLEDGED`

Tip for the Student Affairs screen: an answered request with both `document` and `filled_form` has come back and is ready to check. One with only `document` is still with the requester.

---

7. FixIt: Maintenance tickets

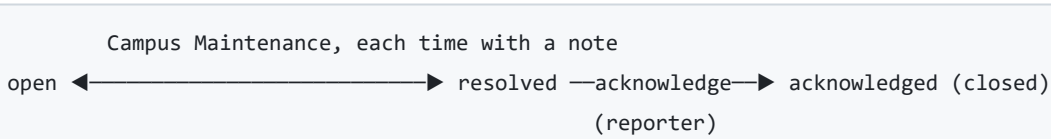
Base path: `/api/v1/fixit`

Users raise tickets for campus problems (a broken fan, a leaking tap). **Campus Maintenance** staff keep the list of issue types, post progress, set turnaround times and resolve tickets. The reporter confirms the fix.

Who can do what

Action	Campus Maintenance	Other office staff	Faculty	Student	Admin
Raise a ticket (with an optional photo)	✅	✅	✅	✅	
Edit / withdraw own ticket, change its photo (while open)	✅	✅	✅	✅	
See tickets	everyone's	own	own	own	everyone's
Update the status with a note	✅				
Delete any ticket (joke or old tickets)	✅				✅
Acknowledge own resolved ticket	✅	✅	✅	✅	
⭐ See issue types	✅	✅	✅	✅	✅
⭐ Create / edit / delete issue types	✅				
⭐ Set turnaround time	✅				

Ticket status



Status	Reporter can	Campus Maintenance can
`open`	edit, withdraw, change the photo	post a progress note, resolve, set the turnaround time
`resolved`	acknowledge	reopen it if it isn't really fixed
`acknowledged`	view	view, delete

What the data looks like

Issue type ⭐

```
{ "issue_type_id": "ITY-...", "name": "Electrical", "description": "Fans, lights, sockets",
  "created_by": "kumar@team1.com", "created_at": "...", "updated_at": "..." }
```

Ticket

```
{
  "ticket_id": "TKT-8206727421084-20448c93",
  "issue_type_id": "ITY-...", "issue_type_name": "Electrical",
  "title": "Fan not working",
  "description": "Room 204, Block B: ceiling fan does not start",
  "status": "open",
  "photo": { "file_id": "PHO-...", "filename": "fan.jpg", "content_type": "image/jpeg", "size_bytes": 182311, "url": "https://..." },
  "tat": { "expected_by": "2026-11-05", "note": "Waiting for the spare part", "updated_by": "kumar@team1.com", "updated_at": "..." },
  "overdue": false,
  "update": { "note": "Electrician assigned", "updated_by": "kumar@team1.com", "updated_by_name": "Kumar Gowda", "updated_at": "..." },
  "reported_by": "asha@team1.com", "reported_by_name": "Asha Kumar", "reported_by_role": "student",
  "reported_by_dept": "CSE", "reported_by_phone_number": "9876543210",
  "created_at": "...", "updated_at": "..."
}
```


- `update` is the latest note from Campus Maintenance, shown to the reporter.
- `overdue` is `true` when `tat.expected_by` has passed and the ticket is still open. Use it for a red badge.
- `photo`, `tat`, `update` and `acknowledged_at` are left out until they exist.

Core APIs

Raise a ticket

`POST /tickets` · **faculty, office staff, student** · form-data → `201` ticket

Field	Required	Notes
`issue_type_id`	✅	from `GET /issue-types`
`title`	✅	3–120 chars
`description`	✅	5–2000 chars. Say where the problem is
`photo`		one JPEG or PNG, up to 5 MB

Track, edit, delete tickets

Method	Endpoint	Who	Notes
GET	`/tickets?status=&issue_type_id=&mine=false&limit=25&cursor=`	everyone	Campus Maintenance and the admin see all tickets, others see their own. `status`: `open`, `resolved`, `acknowledged`. `mine=true` lets Campus Maintenance see only tickets they raised

Method	Endpoint	Who	Notes
GET	`/tickets/{ticket_id}`	reporter, Campus Maintenance, admin	one ticket
PATCH	`/tickets/{ticket_id}`	reporter, while open	form-data, any of: `issue_type_id`, `title`, `description`, `photo` (replaces the old photo), `remove_photo=true`
DELETE	`/tickets/{ticket_id}`	reporter (own ticket, while open), or Campus Maintenance / admin (any ticket)	deletes it with its photo
GET	`/tickets/{ticket_id}/photo`	reporter, Campus Maintenance, admin	download the photo

Update status (Campus Maintenance)

`PUT /tickets/{ticket_id}/status` · JSON → updated ticket

```
{ "status": "resolved", "note": "Capacitor replaced" }
```

`status` is `open` or `resolved`. `note` is required, 1–280 chars, and the reporter sees it.

What you want	Body
Post progress (ticket stays open)	`{ "status": "open", "note": "Electrician assigned" }`
Mark it fixed	`{ "status": "resolved", "note": "Capacitor replaced" }`
Reopen a resolved ticket	`{ "status": "open", "note": "Still not working, checking again" }`

Acknowledge

`POST /tickets/{ticket_id}/acknowledge` · **reporter** · no body. Confirms a resolved ticket is fixed and closes it (`acknowledged`). A ticket that isn't resolved yet returns `409 NOT_RESOLVED`.

⭐ Bonus APIs

⭐ Turnaround time

`PUT /tickets/{ticket_id}/tat` · **Campus Maintenance** · only while `open` · JSON → updated ticket

```
{ "expected_by": "2026-11-05", "note": "Waiting for the spare part" }
```

- `expected_by` is `YYYY-MM-DD` and can't be before today. `note` is optional.
- Sending it again replaces the old value.
- If the date passes while the ticket is still open, the ticket shows `"overdue": true`.
- A resolved ticket returns `409 TICKET_RESOLVED`. Reopen it first.

⭐ Manage issue types (Campus Maintenance)

Method	Endpoint	Body (JSON)
POST	`/issue-types`	`{ "name": "Electrical", "description": "Fans, lights, sockets" }` (more in Sample data). `name` 3–80 chars and unique (ignoring case), `description` optional → `201`
PATCH	`/issue-types/{issue_type_id}`	any of: `name`, `description`
DELETE	`/issue-types/{issue_type_id}`	delete the type

A duplicate name returns `409 ALREADY_EXISTS`. Anyone else returns `403 FORBIDDEN`.

⭐ Load issue types dynamically

Build the "Raise a ticket" dropdown from the API instead of hard-coding it. When Campus Maintenance add a new type, it shows up right away.

Method	Endpoint	Who
GET	`/issue-types?limit=100&cursor=`	everyone
GET	`/issue-types/{issue_type_id}`	everyone

To filter the ticket board by type, use `GET /tickets?issue_type_id=<id>`.

---

8. Error codes

Common to all features

HTTP	`error.code`	Meaning	What to do in the UI
400	`UNKNOWN_INSTITUTION`	Login email domain isn't registered	Check the email
400	`USE_ADMIN_LOGIN`	The admin used the user login	Use the admin login
401	`INVALID_CREDENTIALS`	Wrong email or OTP	Show "wrong email or OTP"
401	`UNAUTHORIZED` / `INVALID_TOKEN` / `TOKEN_EXPIRED`	Missing, bad or expired token	Send the user to login
403	`FORBIDDEN`	This role or department can't do this	Hide the action for this role
404	`NOT_FOUND`	It doesn't exist, or it isn't yours to see	Show "not found"
409	`ALREADY_EXISTS`	Duplicate (user, claim, type name)	Show the message
413	`PAYLOAD_TOO_LARGE` / `FILE_TOO_LARGE` / `IMAGE_TOO_LARGE`	Upload too big	Show the size limit
415	`UNSUPPORTED_FILE_TYPE` / `UNSUPPORTED_IMAGE_TYPE`	Wrong file type	Show the allowed types
422	`VALIDATION_ERROR`	Bad or missing fields; see `details`	Highlight the fields
422	`EMAIL_DOMAIN_MISMATCH`	New user's email isn't your team's domain	Check the email
422	`FIELD_NOT_ALLOWED_FOR_ROLE`	A user edit includes a field the role doesn't have	Remove the field
429	`RATE_LIMITED`	Too many requests or failed logins	Wait for the seconds in the `Retry-After` header
503	`STORAGE_NOT_CONFIGURED`	File uploads aren't set up on the server	Tell the organisers

FindIt

HTTP	`error.code`	Meaning
409	`POST_NOT_ACTIVE`	The post is archived: it can't be claimed or resolved
409	`POST_RESOLVED`	A resolved post can't be reopened with PATCH. Undo the resolution
409	`ALREADY_RESOLVED`	The post already has a resolution. Use PATCH to change it
409	`POST_HAS_PENDING_CLAIMS`	A claim still waits for a decision. Resolve the post before deleting it

HTTP	`error.code`	Meaning
409	`CLAIM_LOCKED`	The claim is already decided or archived and can't be withdrawn
422	`TOO_MANY_IMAGES`	More than 5 images

Notice Hub

HTTP	`error.code`	Meaning
409	`NOTICE_NOT_ACTIVE`	Doubts can only be asked on active notices
409	`NOTICE_ALREADY_PUBLISHED`	`publish_at` can only change while the notice is scheduled
409	`DOUBT_ANSWERED`	An answered doubt can't be edited
409	`ALREADY_ANSWERED`	The doubt already has a response. Use PATCH

DocDesk

HTTP	`error.code`	Meaning
409	`DOC_TYPE_ARCHIVED`	This document type takes no new requests
409	`REQUEST_NOT_PENDING`	Already answered or acknowledged (bulk: see `details.request_ids`)
409	`ALREADY_ANSWERED`	Use PATCH to change the answer
409	`NOT_ANSWERED`	Nothing to acknowledge or fill in yet
409	`NO_DOCUMENT`	The answer has no document to fill in
409	`ALREADY_ACKNOWLEDGED`	The request is closed

FixIt

HTTP	`error.code`	Meaning
409	`TICKET_NOT_OPEN`	The reporter can only edit, withdraw or change the photo while the ticket is open
409	`TICKET_RESOLVED`	A resolved ticket can't get a turnaround time. Reopen it first
409	`NOT_RESOLVED`	Nothing to acknowledge yet
409	`ALREADY_ACKNOWLEDGED`	The ticket is closed

---

9. Limits

What	Limit
JSON body	16 KB
Page size	1–100 (default 25)
Requests per user	300 per minute
FindIt images	up to 5 per post, 5 MB each, JPEG / PNG / WebP
Notice attachment	1 per notice, 10 MB, PDF / JPEG / PNG
DocDesk document	10 MB, PDF / Word (.docx)
DocDesk filled-in form	10 MB, PDF / Word (.docx) / JPEG / PNG
FixIt photo	1 per ticket, 5 MB, JPEG / PNG
File `url` lifetime	about 15 minutes (download endpoints always work)

The API stores text exactly as sent, so **escape it when you display it**. React does this for you; never put user text into `dangerouslySetInnerHTML`.

---

10. All endpoints at a glance

⭐ = bonus

Accounts

	Method	Endpoint	Who
	POST	`/api/v1/auth/admin/login`	admin
	POST	`/api/v1/auth/login`	student, faculty, office staff
	GET	`/api/v1/users/me`	student, faculty, office staff
	GET	`/api/v1/admin/me`	admin
	POST	`/api/v1/admin/users`	admin
	GET	`/api/v1/admin/users`	admin
	GET	`/api/v1/admin/users/{email_id}`	admin
	PATCH	`/api/v1/admin/users/{email_id}`	admin
	POST	`/api/v1/admin/users/{email_id}/regenerate-otp`	admin
	DELETE	`/api/v1/admin/users/{email_id}`	admin

FindIt (`/api/v1/findit`)

	Method	Endpoint	Who
	POST	`/posts`	office staff
	GET	`/posts`	everyone
	GET	`/posts/{post_id}`	everyone
	PATCH	`/posts/{post_id}`	office staff
	DELETE	`/posts/{post_id}`	office staff
	POST	`/posts/{post_id}/claims`	everyone
	GET	`/posts/{post_id}/claims`	everyone
	GET	`/claims/me`	everyone
	GET	`/claims/{claim_id}`	claimant, office staff
	DELETE	`/claims/{claim_id}`	claimant
	POST	`/posts/{post_id}/resolution`	office staff
	GET	`/posts/{post_id}/resolution`	office staff, approved claimant
	PATCH	`/posts/{post_id}/resolution`	office staff
	DELETE	`/posts/{post_id}/resolution`	office staff

	Method	Endpoint	Who
⭐	POST	`/lost-requests`	faculty, office staff, student
⭐	GET	`/lost-requests`	faculty, office staff, student
⭐	GET	`/lost-requests/{request_id}`	owner, office staff
⭐	PATCH	`/lost-requests/{request_id}`	owner
⭐	DELETE	`/lost-requests/{request_id}`	owner
⭐	PATCH	`/posts/{post_id}` with `status`, and `GET /posts?status=archived`	office staff

Notice Hub (`/api/v1/noticehub`)

	Method	Endpoint	Who
	POST	`/notices`	admin, faculty, office staff
	GET	`/notices`	everyone
	GET	`/notices/{notice_id}`	everyone
	PATCH	`/notices/{notice_id}`	publisher
	DELETE	`/notices/{notice_id}`	publisher, admin
	POST	`/notices/{notice_id}/doubts`	faculty, office staff, student
	GET	`/notices/{notice_id}/doubts`	everyone
	GET	`/doubts/{doubt_id}`	everyone
	PATCH	`/doubts/{doubt_id}`	asker
	DELETE	`/doubts/{doubt_id}`	asker, admin
	POST	`/doubts/{doubt_id}/response`	publisher, admin
	GET	`/doubts/{doubt_id}/response`	everyone
	PATCH	`/doubts/{doubt_id}/response`	responder, admin
	DELETE	`/doubts/{doubt_id}/response`	responder, admin
⭐	POST / PATCH	`/notices` with `publish_at`, and `GET /notices?status=scheduled`	publisher
⭐	PATCH	`/notices/{notice_id}` with `status`, and `GET /notices?status=archived`	publisher
⭐	PATCH	`/notices/{notice_id}` with `pinned`, and `GET /notices?pinned=true`	publisher
⭐	GET	`/notices/{notice_id}/attachment`	everyone

DocDesk (`/api/v1/docdesk`)

	Method	Endpoint	Who
	POST	`/requests`	faculty, office staff, student
	GET	`/requests`	everyone
	GET	`/requests/{request_id}`	requester, Student Affairs, admin
	PATCH	`/requests/{request_id}`	requester
	DELETE	`/requests/{request_id}`	requester
	POST	`/requests/{request_id}/answer`	Student Affairs
	PATCH	`/requests/{request_id}/answer`	Student Affairs
	DELETE	`/requests/{request_id}/answer`	Student Affairs
	POST	`/requests/{request_id}/acknowledge`	requester
⭐	GET	`/doc-types`	everyone
⭐	GET	`/doc-types/{doc_type_id}`	everyone
⭐	POST	`/doc-types`	Student Affairs
⭐	PATCH	`/doc-types/{doc_type_id}`	Student Affairs
⭐	DELETE	`/doc-types/{doc_type_id}`	Student Affairs
⭐	PUT	`/requests/{request_id}/tat`	Student Affairs
⭐	POST	`/requests/bulk-tat`	Student Affairs
⭐	GET	`/requests/{request_id}/document`	requester, Student Affairs, admin
⭐	PUT	`/requests/{request_id}/filled-form`	requester
⭐	GET	`/requests/{request_id}/filled-form`	requester, Student Affairs, admin
⭐	DELETE	`/requests/{request_id}/filled-form`	requester

FixIt (`/api/v1/fixit`)

	Method	Endpoint	Who
	POST	`/tickets`	faculty, office staff, student
	GET	`/tickets`	everyone
	GET	`/tickets/{ticket_id}`	reporter, Campus Maintenance, admin
	PATCH	`/tickets/{ticket_id}`	reporter
	DELETE	`/tickets/{ticket_id}`	reporter, Campus Maintenance, admin
	GET	`/tickets/{ticket_id}/photo`	reporter, Campus Maintenance, admin
	PUT	`/tickets/{ticket_id}/status`	Campus Maintenance
	POST	`/tickets/{ticket_id}/acknowledge`	reporter
⭐	PUT	`/tickets/{ticket_id}/tat`	Campus Maintenance

	Method	Endpoint	Who
⭐	GET	`/issue-types`	everyone
⭐	GET	`/issue-types/{issue_type_id}`	everyone
⭐	POST	`/issue-types`	Campus Maintenance
⭐	PATCH	`/issue-types/{issue_type_id}`	Campus Maintenance
⭐	DELETE	`/issue-types/{issue_type_id}`	Campus Maintenance